

Predicción de conversión en e-commerce parafarmacéutico (Google Analytics)

por Alejandro Sanz Serrano

Farmacéutico + Máster en Data Cience & Business Analytics

En el entorno actual del comercio electrónico, uno de los principales retos para las empresas del sector parafarmacéutico es identificar qué usuarios presentan una mayor probabilidad de conversión durante su navegación en la web. La optimización de este proceso permite mejorar la eficiencia de las campañas de marketing, personalizar la experiencia del usuario y maximizar el retorno de la inversión.

Este proyecto aborda este problema mediante el análisis de datos de comportamiento web basados en métricas de Google Analytics, representativos de un entorno de e-commerce.

El objetivo principal es desarrollar un modelo predictivo capaz de estimar la probabilidad de conversión de un usuario durante una sesión, con el fin de:

1. Identificar usuarios con alta intención de compra
2. Priorizar acciones comerciales y de marketing
3. Optimizar la asignación de recursos
4. Mejorar la tasa de conversión

Datos: El análisis se realiza sobre un dataset de una parafarmacia anónima que realiza venta online de comportamiento de usuarios en entornos de e-commerce, basado en variables típicas de Google Analytics (tiempo de navegación, número de páginas visitadas, tasa de rebote, etc.).

Enfoque metodológico: El proyecto sigue un pipeline completo de análisis de datos:

1. Análisis exploratorio de datos (EDA)
2. Preprocesamiento y transformación de variables
3. Modelización predictiva (Regresión Logística, Random Forest, Redes Neuronales)
4. Evaluación mediante métricas robustas (ROC-AUC)
5. Optimización de hiperparámetros
6. Interpretación de resultados y análisis de variables clave
7. Aplicación práctica a negocio
8. Valor aportado

El modelo desarrollado permite pasar de un enfoque reactivo a uno proactivo, facilitando la identificación de usuarios con mayor probabilidad de conversión en tiempo real. Esto abre la puerta a múltiples aplicaciones en un entorno empresarial, como la segmentación avanzada de clientes, la personalización de la experiencia de usuario o la optimización de campañas de marketing.

Consideraciones: Se incluyen limitaciones relevantes del modelo, como la ausencia de identificadores únicos de usuario o la falta de información temporal completa, lo que refuerza el carácter realista y crítico del análisis realizado.

Importación de librerías

In [2]:

```
# Manipulación de datos
import pandas as pd          # Para trabajar con dataframes (cargar, limpiar, transformar datos)
import numpy as np           # Para operaciones numéricas

# Visualización
import matplotlib.pyplot as plt  # Gráficos básicos
import seaborn as sns           # Gráficos más avanzados y bonitos

# Configuración visual (OJO: opcional pero muy recomendable)
sns.set(style="whitegrid")      # Estilo de gráficos
plt.rcParams["figure.figsize"] = (8, 5)  # Tamaño por defecto de las figuras

# Preprocesado y modelado
from sklearn.model_selection import train_test_split  # División train/test
from sklearn.preprocessing import StandardScaler      # Estandarización de variables
from sklearn.preprocessing import LabelEncoder        # Codificación de variables categóricas
from sklearn.preprocessing import OneHotEncoder        # Alternativa para categóricas

# Modelos que me solicita el enunciado
from sklearn.linear_model import LogisticRegression  # Modelo lineal
from sklearn.ensemble import RandomForestClassifier  # Modelo de árboles
from sklearn.neural_network import MLPClassifier    # Red neuronal

# Evaluación
from sklearn.metrics import roc_auc_score, roc_curve  # Métrica principal (ROC)
from sklearn.metrics import classification_report      # Precision, recall, etc.
```

```
# Optimización
from sklearn.model_selection import GridSearchCV # Búsqueda de hiperparámetros
```

Bloque 1 - Carga de datos

```
In [3]: # Carga del dataset
df = pd.read_csv("online_shoppers_intention.csv")

# Visualización inicial de las primeras filas
df.head()
```

Out[3]:

	Administrative	Administrative_Duration	Informational	Informational_Duration	ProductRelated	ProductRelated_Duration	BounceR
0	0.0	0.0	0.0	0.0	1.0	0.000000	
1	0.0	0.0	0.0	0.0	2.0	64.000000	
2	0.0	-1.0	0.0	-1.0	1.0	-1.000000	
3	0.0	0.0	0.0	0.0	2.0	2.666667	
4	0.0	0.0	0.0	0.0	10.0	627.500000	

Bloque 2 - Inspección inicial

```
In [4]: # Dimensiones del dataset
df.shape

# Información general (tipos de datos y nulos)
df.info()

# Estadísticas descriptivas
df.describe()

# Distribución del target
df["Revenue"].value_counts(normalize=True)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12330 entries, 0 to 12329
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Administrative         12316 non-null  float64
1   Administrative_Duration 12316 non-null  float64
2   Informational           12316 non-null  float64
3   Informational_Duration  12316 non-null  float64
4   ProductRelated          12316 non-null  float64
5   ProductRelated_Duration 12316 non-null  float64
6   BounceRates             12316 non-null  float64
7   ExitRates              12316 non-null  float64
8   PageValues             12330 non-null  float64
9   SpecialDay             12330 non-null  float64
10  Month                  12330 non-null  object
11  OperatingSystems       12330 non-null  int64
12  Browser                12330 non-null  int64
13  Region                 12330 non-null  int64
14  TrafficType            12330 non-null  int64
15  VisitorType            12330 non-null  object
16  Weekend                12330 non-null  bool
17  Revenue                12330 non-null  bool
dtypes: bool(2), float64(10), int64(4), object(2)
memory usage: 1.5+ MB
```

```
Out[4]: Revenue
False    0.845255
True     0.154745
Name: proportion, dtype: float64
```

El dataset contiene 12.330 filas, donde cada una representa una sesión de usuario en una web de comercio electrónico, y 18 variables en total. La mayoría de las variables son numéricas y están relacionadas con el comportamiento durante la sesión (número de páginas visitadas, tiempo de navegación, tasas de rebote, etc.), aunque también hay variables categóricas como el mes o el tipo de visitante, y variables booleanas como Weekend y Revenue.

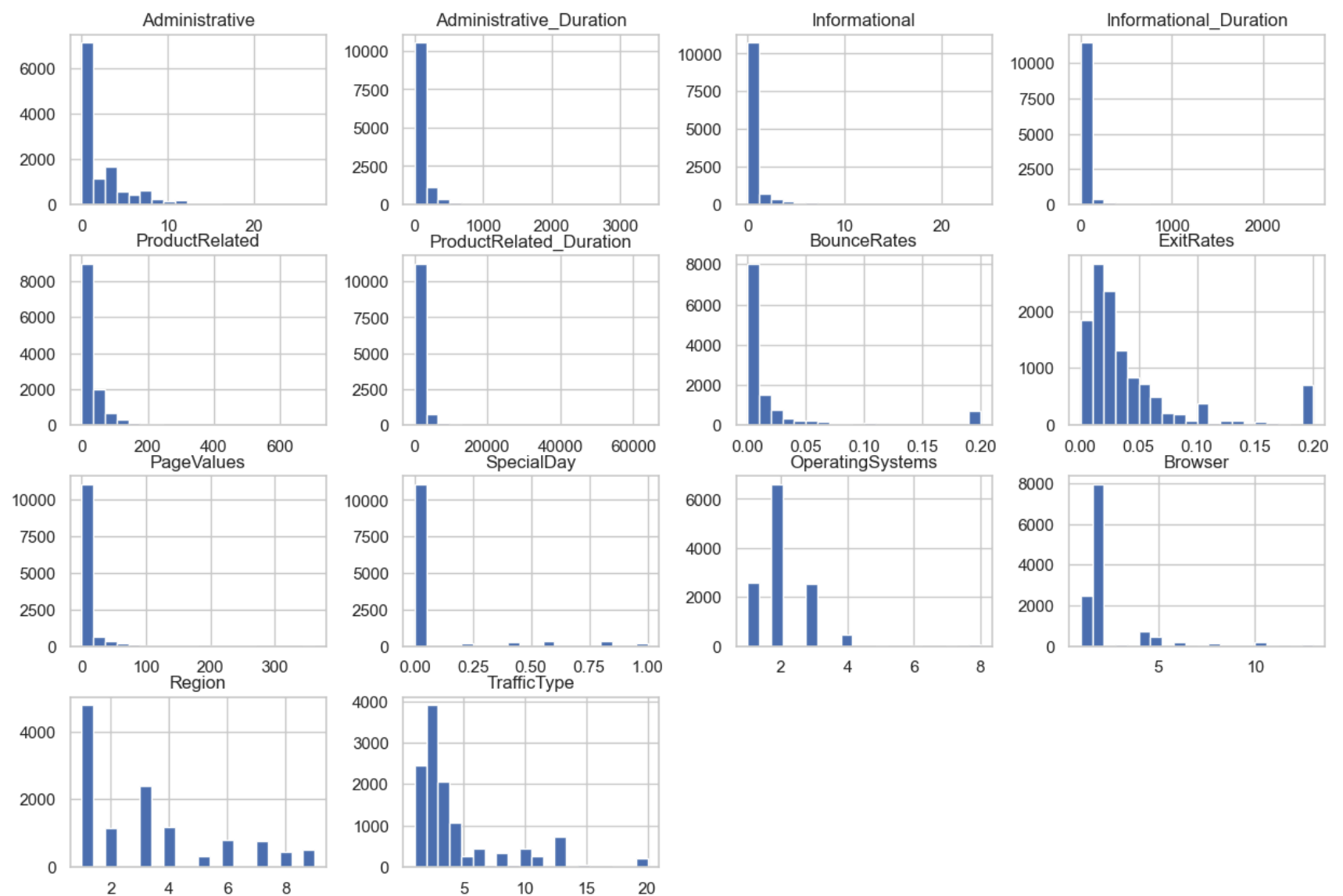
Se observa un número muy pequeño de valores faltantes (14 registros), por lo que se pueden eliminar sin impacto relevante en el análisis usando dropna().

En cuanto a la variable objetivo (Revenue), el dataset está desbalanceado, con aproximadamente un 85% de sesiones sin compra frente a un 15% con compra. Esto será importante a la hora de elegir las métricas de evaluación.

Además, algunas variables como OperatingSystems, Browser, Region y TrafficType están codificadas como números, aunque en realidad representan categorías, por lo que habrá que tratarlas adecuadamente más adelante.

Bloque 3 - Análisis exploratorio de datos (EDA)

```
In [5]: # Distribución de variables numéricas
df.hist(bins=20, figsize=(15,10))
plt.show()
```



La mayoría de las variables numéricas presentan distribuciones fuertemente sesgadas a la derecha, con una gran concentración de valores en torno a cero. Esto sugiere que la mayoría de las sesiones presentan una baja interacción, mientras que un pequeño grupo de usuarios muestra una actividad mucho más intensa.

Se observan outliers especialmente en las variables de duración, lo cual puede corresponder a usuarios que pasan mucho tiempo navegando. Estos valores no se consideran necesariamente erróneos, ya que pueden aportar información relevante sobre el comportamiento del usuario.

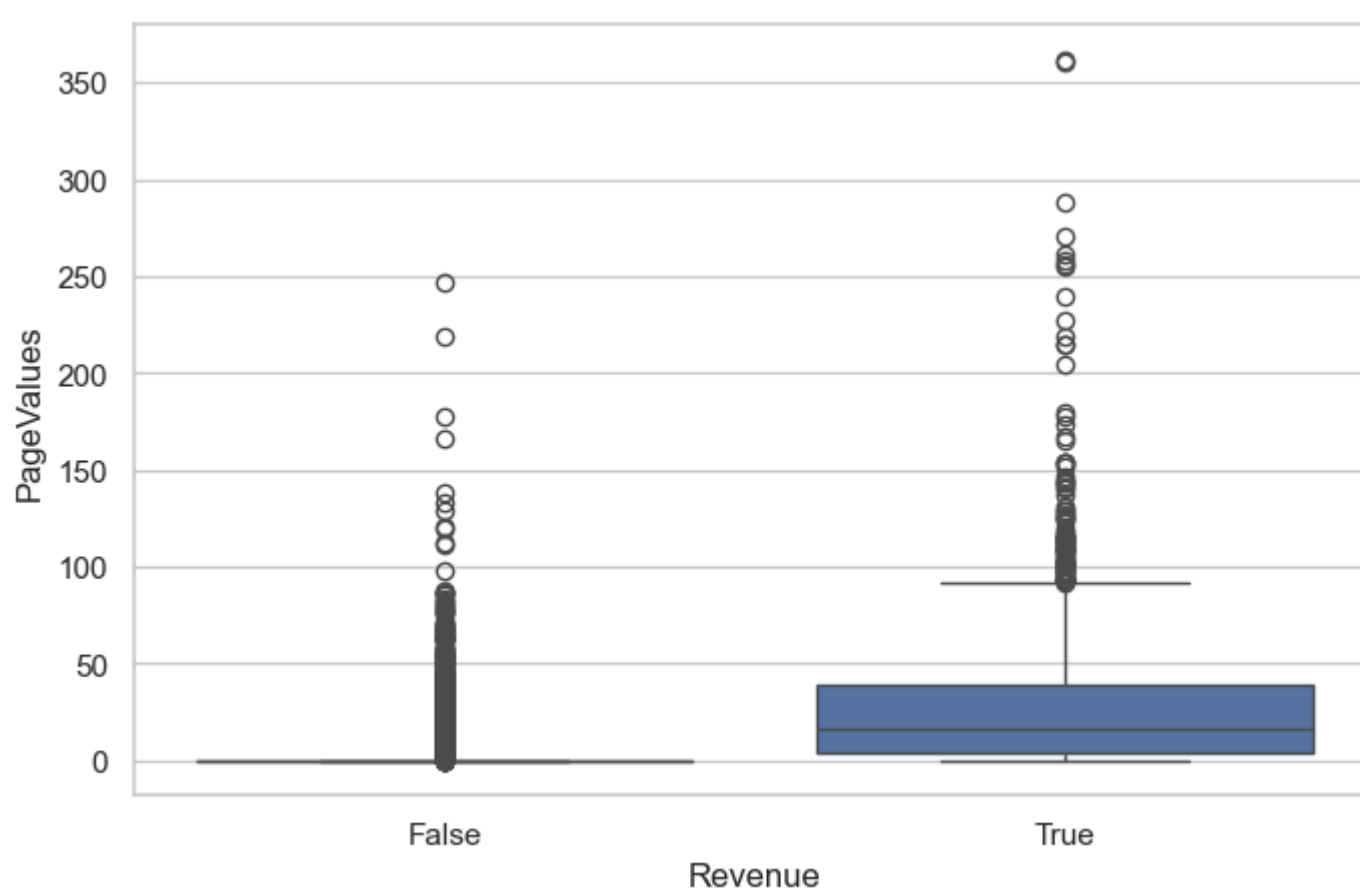
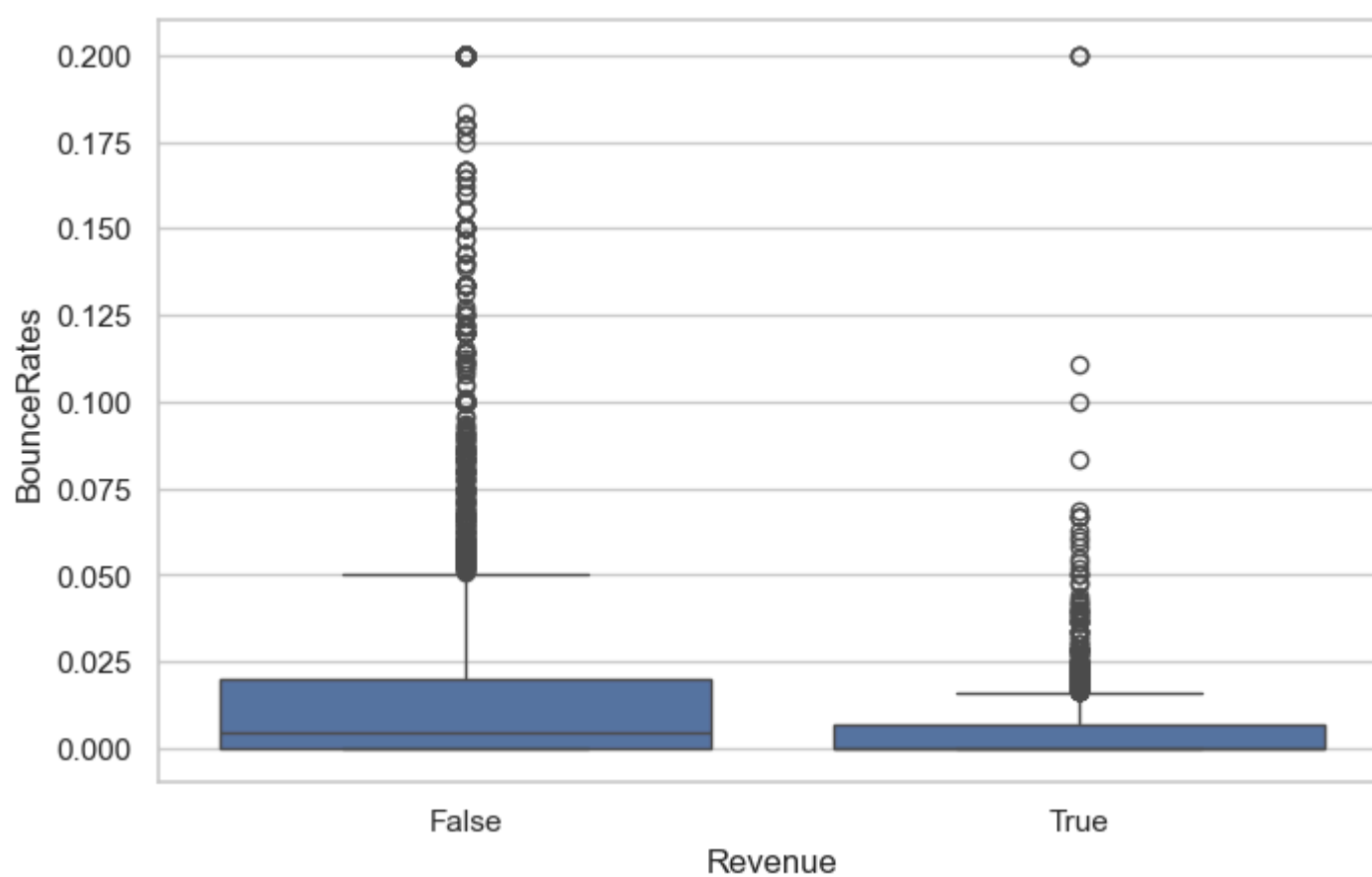
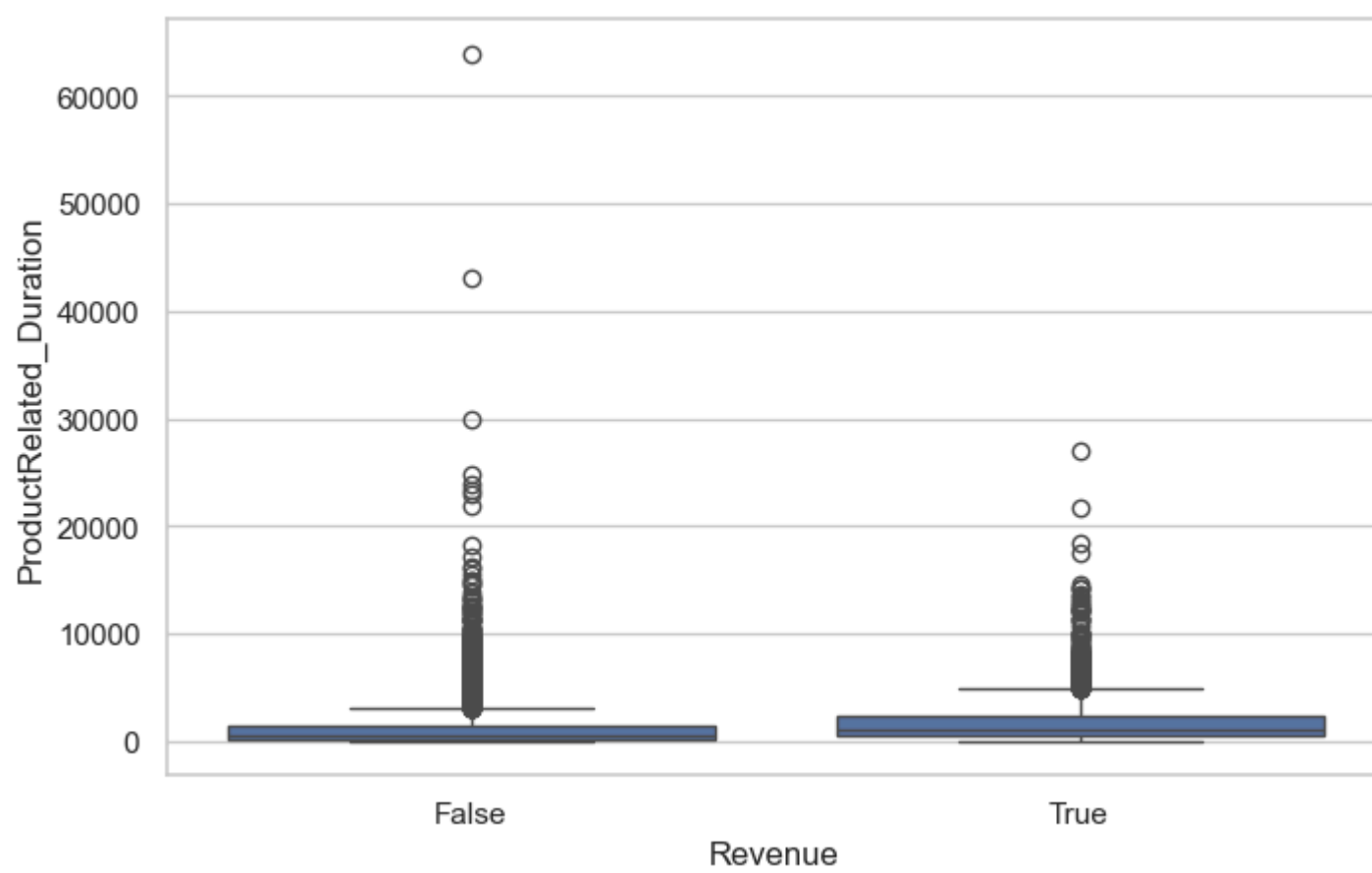
Variables como BounceRates y ExitRates muestran patrones coherentes con el comportamiento esperado, mientras que PageValues presenta muchos valores nulos junto con algunos valores elevados, lo que sugiere que podría ser una variable altamente predictiva, aunque será necesario analizarla con más detalle.

3.1. Relación con la variable objetivo (Revenue)

```
In [6]: # Boxplot de variables clave vs Revenue
sns.boxplot(x="Revenue", y="ProductRelated_Duration", data=df)
plt.show()

sns.boxplot(x="Revenue", y="BounceRates", data=df)
plt.show()

sns.boxplot(x="Revenue", y="PageValues", data=df)
plt.show()
```



Para entender mejor qué factores influyen en la compra, se han comparado varias variables clave en función de si la sesión termina en conversión (Revenue = True) o no.

En el caso de ProductRelated_Duration, se observa que los usuarios que realizan una compra tienden a pasar más tiempo navegando por páginas de producto. Esto tiene bastante sentido desde el punto de vista del comportamiento: cuanto más

tiempo dedica un usuario a explorar productos, mayor es su interés y, por tanto, la probabilidad de compra.

Por otro lado, la variable BounceRates muestra el patrón contrario. Las sesiones que terminan en compra presentan valores más bajos, lo que indica que estos usuarios no abandonan la página rápidamente. En cambio, los usuarios que no compran tienden a tener tasas de rebote más altas, lo que sugiere una menor implicación con el contenido.

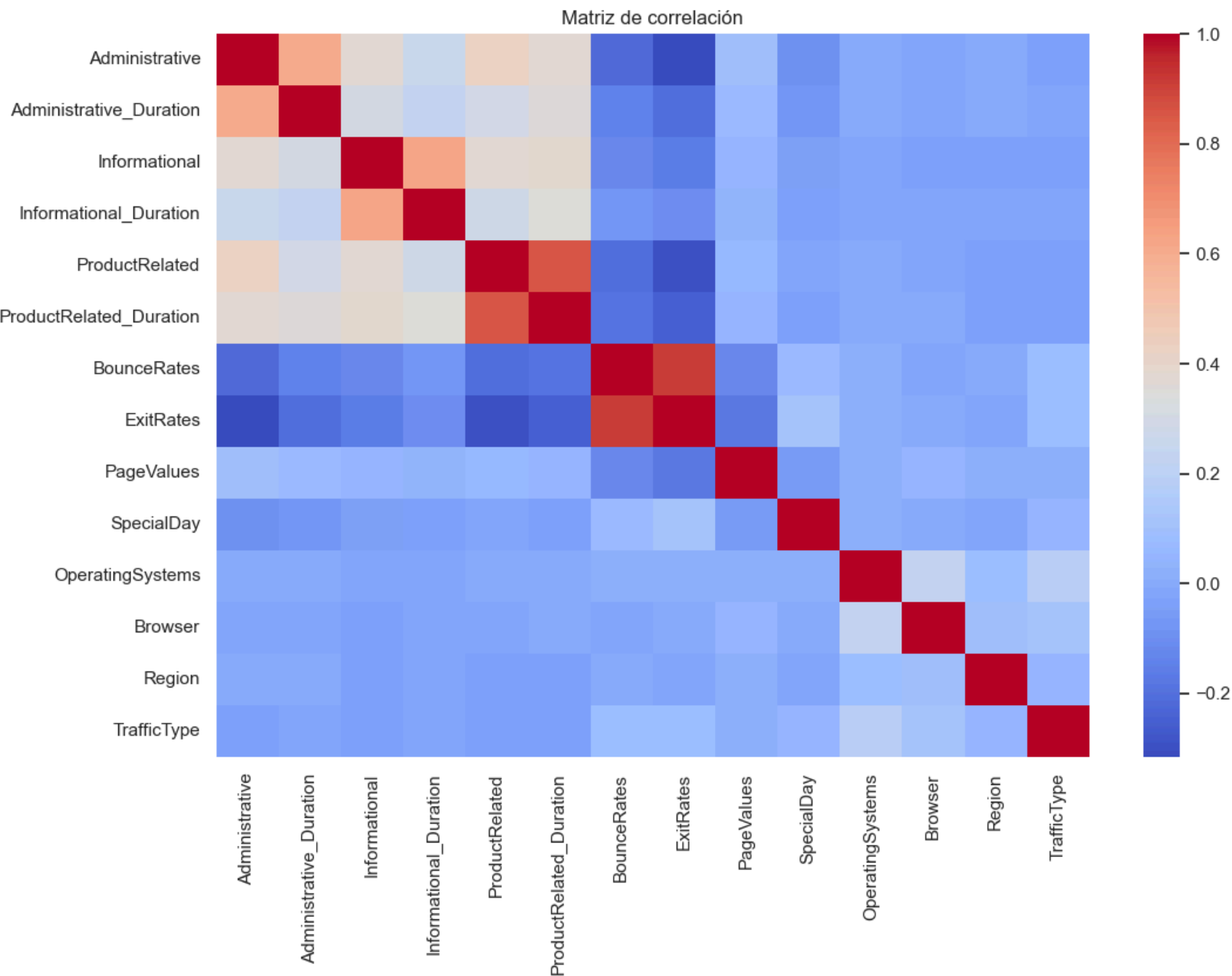
La variable PageValues es especialmente interesante. En las sesiones sin compra, la mayoría de los valores son cercanos a cero, mientras que en las sesiones con compra aparecen valores claramente más elevados. Esto indica que esta variable tiene una gran capacidad para diferenciar entre usuarios que compran y los que no. Sin embargo, esta separación tan clara también hace sospechar que podría existir cierto data leakage, ya que PageValues parece estar muy directamente relacionada con la conversión. Será importante analizar con más detalle su definición antes de utilizarla en el modelo. Si PageValues está calculado utilizando información posterior a la conversión, introduciría data leakage (es decir, el valor es alto cuando alguien compra, el modelo está "haciendo trampa"). En ese caso, lo más prudente sería excluir esta variable del modelo o analizar su definición en detalle antes de utilizarla. En este caso, en vez de eliminar la variable, se va a probar el modelo con y sin ella y, si el modelo depende demasiado de PageValues, podría no generalizar bien y es mejor eliminarla.

3.2. Análisis de correlaciones

```
In [7]: # Selección solo de variables numéricas para la correlación (OJO: Las categóricas no tienen sentido aquí)
df_numeric = df.select_dtypes(include=["float64", "int64"])

# Calcular la matriz de correlación
corr_matrix = df_numeric.corr()

# Visualización con heatmap (me parece la mejor opción: se visualizan muchas correlaciones a la vez
# en una matriz con varios colores para indicar la intensidad - rojo: positiva, azul: negativa, colores
# suaves: poca correlación)
plt.figure(figsize=(12,8))
sns.heatmap(corr_matrix, cmap="coolwarm", annot=False)
plt.title("Matriz de correlación")
plt.show()
```



A partir de la matriz de correlación se observan varias relaciones interesantes entre las variables numéricas. En primer lugar, destacan las altas correlaciones entre variables de conteo y sus correspondientes variables de duración, como ProductRelated y ProductRelated_Duration, Informational y Informational_Duration, o Administrative y Administrative_Duration. Esto es coherente, ya que un mayor número de páginas visitadas suele implicar un mayor tiempo de navegación. Estas correlaciones podrían introducir problemas de multicolinealidad en modelos lineales.

También se observa una correlación positiva entre BounceRates y ExitRates, lo que tiene sentido desde el punto de vista del comportamiento del usuario, ya que las páginas con altas tasas de rebote suelen coincidir con mayores tasas de salida.

Por otro lado, variables como PageValues presentan una baja correlación con el resto, lo que sugiere que aportan información diferente y potencialmente relevante para el modelo.

Finalmente, las variables categóricas codificadas como números (OperatingSystems, Browser, Region, TrafficType) no muestran correlaciones significativas, lo cual es esperable por su carácter categórico. Estas variables no muestran correlación lineal relevante, pero esto puede deberse a que son categóricas codificadas como números. Antes de decidir eliminarlas, sería necesario transformarlas adecuadamente (por ejemplo, mediante one-hot encoding) y evaluar su impacto en el modelo.

Las variables de conteo y duración están altamente correlacionadas, por lo que podría ser conveniente evaluar si ambas aportan información adicional o si alguna puede eliminarse para evitar multicolinealidad en modelos lineales.

Bloque 4 - Procesado de datos

4.1. Tratamiento de valores nulos

Tras analizar el dataset, se observa la presencia de un número muy reducido de valores nulos (14 registros sobre un total de más de 12.000 observaciones). Dado que este porcentaje es despreciable, se opta por eliminar dichas filas mediante el método dropna(), ya que su eliminación no afectará de forma significativa al análisis ni al rendimiento del modelo.

No se considera necesario aplicar técnicas de imputación (como fillna()), ya que estas suelen utilizarse cuando el volumen de datos faltantes es mayor y su eliminación podría suponer una pérdida relevante de información. En este caso, imputar valores podría introducir ruido innecesario en el dataset.

Tampoco se emplean métodos de reemplazo (replace()), ya que estos no están específicamente diseñados para el tratamiento de valores nulos, sino para la sustitución de valores concretos dentro de las variables.

```
In [8]: # Eliminación de valores nulos
df = df.dropna()
```

4.2. Tratamiento de variables categóricas (por one-hot encoding)

Las variables booleanas no requieren transformación, ya que pueden ser interpretadas directamente por los modelos. En cambio, las variables categóricas deberán ser transformadas a formato numérico mediante técnicas como one-hot encoding para evitar introducir relaciones de orden inexistentes.

```
In [9]: pd.get_dummies(df)
```

Out[9]:

	Administrative	Administrative_Duration	Informational	Informational_Duration	ProductRelated	ProductRelated_Duration
0	0.0	0.0	0.0	0.0	1.0	0.0
1	0.0	0.0	0.0	0.0	2.0	0.0
2	0.0	-1.0	0.0	-1.0	1.0	0.0
3	0.0	0.0	0.0	0.0	2.0	0.0
4	0.0	0.0	0.0	0.0	10.0	0.0
...	...	...	...	...	...	...
12325	3.0	145.0	0.0	0.0	53.0	0.0
12326	0.0	0.0	0.0	0.0	5.0	0.0
12327	0.0	0.0	0.0	0.0	6.0	0.0
12328	4.0	75.0	0.0	0.0	15.0	0.0
12329	0.0	0.0	0.0	0.0	3.0	0.0

12316 rows × 29 columns



```
In [10]: # Aplicar one-hot encoding
df_encoded = pd.get_dummies(df, drop_first=True)
```


Apunte: se utiliza `drop_first=True` para evitar la multicolinealidad derivada del llamado dummy variable trap. Si se incluyeran todas las categorías, una de ellas podría expresarse como combinación lineal de las demás. Por ello, se elimina una categoría de referencia, de modo que el resto de variables indican la pertenencia a las otras categorías.

Por ilustrar el ejemplo de manera más sencilla, si tuviéramos una variable como el sexo (masculino/femenino), el one-hot encoding generaría dos columnas: una para masculino y otra para femenino. Sin embargo, aplicando `drop_first=True`, se obtiene una única columna que representa la presencia o ausencia de una de las categorías. En este caso, si se mantiene la categoría "femenino", el valor 1 indicaría femenino y el valor 0 masculino. De este modo, se pasa de tener dos columnas a una sola, evitando redundancias y posibles problemas de multicolinealidad.

Además, este procedimiento permite convertir variables categóricas a formato numérico sin introducir un orden artificial entre categorías, lo cual es fundamental para el correcto funcionamiento de los modelos. En este ejercicio, el uso de one-hot encoding permitirá que los modelos de machine learning puedan interpretar correctamente variables como el mes, el tipo de visitante o el tipo de tráfico, evitando que se establezcan relaciones erróneas entre categorías y mejorando así la capacidad predictiva del modelo.

4.3 Escalado de variables

El escalado de variables consiste en transformar las variables numéricas para que todas tengan una escala comparable. Esto es importante porque algunas variables pueden tener valores mucho más altos que otras (por ejemplo, una variable con valores en torno a 3000 frente a otra entre 0 y 1), lo que podría hacer que el modelo les dé más peso del que realmente tienen.

En este caso, se aplicará un escalado de tipo `StandardScaler`, que transforma las variables para que tengan media 0 y desviación estándar 1. Este proceso es especialmente relevante para modelos como la regresión logística y las redes neuronales, ya que son sensibles a la magnitud de las variables.

Por el contrario, modelos basados en árboles como Random Forest no requieren escalado, aunque se aplicará igualmente para mantener coherencia en el preprocesado.

```
In [11]: # Separación de variables
X = df_encoded.drop("Revenue", axis=1)
y = df_encoded["Revenue"]

# División en train/test
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
In [12]: from sklearn.preprocessing import StandardScaler

# Inicializar el scaler
scaler = StandardScaler()

# Ajustar el scaler SOLO con Los datos de entrenamiento
X_train_scaled = scaler.fit_transform(X_train)

# Aplicar la transformación al conjunto de test (sin volver a ajustar)
X_test_scaled = scaler.transform(X_test)
```

```
In [13]: # Ver dimensiones
print("X_train:", X_train.shape)
print("X_test:", X_test.shape)

# Ver datos escalados (primeras filas)
print(X_train_scaled[:5])
```



```

X_train: (9852, 26)
X_test: (2464, 26)
[[ 0.50021261  1.04166025 -0.39540284 -0.24356156 -0.22019446 -0.30269006
 -0.37280244 -0.66723439 -0.31756251 -0.30844509  0.96013173 -0.20916732
  1.18113412 -0.51555114  1.82393687 -0.40800654 -0.12264682 -0.19106832
 -0.15426326 -0.42778162 -0.61520767  1.79021383 -0.2149257  -0.19809217
 -0.08699431  0.41172281]
[-0.69847261 -0.45928051 -0.39540284 -0.24356156 -0.26602237  0.32749768
 -0.19646959 -0.27738002 -0.31756251  2.71423498 -1.24558537 -0.20916732
 -0.89993988 -0.51555114 -0.54826459 -0.40800654 -0.12264682 -0.19106832
 -0.15426326 -0.42778162  1.62546737 -0.55859249 -0.2149257  -0.19809217
 -0.08699431  0.41172281]
[-0.09913  -0.3743216  -0.39540284 -0.24356156 -0.42642006 -0.58607428
 -0.15455277 -0.29126166 -0.31756251 -0.30844509 -0.14272682 -0.20916732
  2.42977853 -0.76516829 -0.54826459 -0.40800654 -0.12264682 -0.19106832
 -0.15426326  2.33764132 -0.61520767 -0.55859249 -0.2149257  -0.19809217
 -0.08699431  0.41172281]
[-0.69847261 -0.45928051 -0.39540284 -0.24356156 -0.28893632 -0.32300912
 -0.45321022 -0.64034455  4.5396655  -0.30844509 -0.14272682 -0.20916732
 -0.89993988 -0.51555114 -0.54826459 -0.40800654 -0.12264682 -0.19106832
 -0.15426326 -0.42778162 -0.61520767  1.79021383 -0.2149257  -0.19809217
 -0.08699431  0.41172281]
[ 1.39922652  0.10031552 -0.39540284 -0.24356156 -0.6097317  -0.59006854
  0.01136805 -0.19235486 -0.31756251 -0.30844509  0.96013173 -0.20916732
 -0.48372508  0.23330032  1.82393687 -0.40800654 -0.12264682  5.23373011
 -0.15426326 -0.42778162 -0.61520767 -0.55859249 -0.2149257  -0.19809217
 -0.08699431 -2.42881857]]

```

Bloque 5 - Modelado

5.1. Regresión logística

Se entrena un modelo de regresión logística como baseline, utilizando los datos previamente escalados. Este modelo permite estimar la probabilidad de que una sesión termine en compra.

Para evaluar su rendimiento, se utiliza la métrica ROC-AUC, adecuada en contextos de desbalanceo de clases, ya que mide la capacidad del modelo para distinguir entre sesiones con y sin conversión.

```

In [14]: from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score

# Inicializar el modelo
log_model = LogisticRegression(max_iter=1000)

# Entrenar el modelo con los datos escalados
log_model.fit(X_train_scaled, y_train)

# Predicción de probabilidades (importante para ROC)
y_pred_proba = log_model.predict_proba(X_test_scaled)[: , 1]

# Cálculo de ROC-AUC
roc_auc = roc_auc_score(y_test, y_pred_proba)

print("ROC-AUC:", roc_auc)

```

ROC-AUC: 0.8890100135290775

Con un ROC-AUC cercano al 0.9, el modelo es bastante bueno diferenciando entre usuarios que compran y los que no. Ahora, es importante saber si el modelo está o no inflado por PageValues, tal y como se estableció anteriormente. Para evaluar el impacto de PageValues, se construirá un segundo conjunto de datos excluyendo dicha variable, manteniendo el resto del pipeline igual para garantizar una comparación justa.

```

In [15]: # 1. Crear dataset sin PageValues

X = df_encoded.drop(["Revenue", "PageValues"], axis=1)
y = df_encoded["Revenue"]

# 2. Train/Test split
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# 3. Escalado de variables

from sklearn.preprocessing import StandardScaler

```



```

scaler = StandardScaler()

# Ajustar solo con train
X_train_scaled = scaler.fit_transform(X_train)

# Aplicar al test
X_test_scaled = scaler.transform(X_test)

# 4. Modelo de regresión logística

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score

# Inicializar el modelo
log_model = LogisticRegression(max_iter=1000)

# Entrenar
log_model.fit(X_train_scaled, y_train)

# Predicción de probabilidades
y_pred_proba = log_model.predict_proba(X_test_scaled)[:, 1]

# Evaluación
roc_auc = roc_auc_score(y_test, y_pred_proba)

print("ROC-AUC sin PageValues:", roc_auc)

```

ROC-AUC sin PageValues: 0.7414248784143318

Al comparar los resultados del modelo con y sin la variable PageValues, se observa una diferencia significativa en el rendimiento. El modelo que incluye esta variable alcanza un ROC-AUC cercano a 0.9, mientras que al eliminarla el rendimiento desciende hasta aproximadamente 0.74. Esta diferencia sugiere que PageValues contiene una gran cantidad de información predictiva. Sin embargo, dado que esta variable podría estar calculada a partir de información relacionada con la conversión, su uso podría introducir data leakage.

Por ello, aunque su inclusión mejora notablemente el rendimiento del modelo, se considera que los resultados sin PageValues ofrecen una estimación más realista del comportamiento del modelo en un entorno de producción.

5.2 Red neuronal (MLPClassifier)

Se entrena un modelo de red neuronal multicapa (MLPClassifier) como alternativa más flexible a la regresión logística. Este tipo de modelo permite capturar relaciones no lineales entre variables, lo que puede resultar útil en problemas de comportamiento de usuario.

Para su entrenamiento, se utilizan los datos previamente escalados, ya que este tipo de modelos son sensibles a la magnitud de las variables.

```

In [16]: from sklearn.neural_network import MLPClassifier
from sklearn.metrics import roc_auc_score

# Inicializar el modelo
mlp_model = MLPClassifier(                                # Aprende relaciones no lineales
    hidden_layer_sizes=(100,),                             # Define la arquitectura - 1 capa oculta con 100 neuronas,
    max_iter=300,                                           # Número máximo de iteraciones (cuántas veces ve los datos)
    random_state=42                                         # Fija la semilla aleatoria del modelo = resultados reproducibles
)

# Entrenarlo
mlp_model.fit(X_train_scaled, y_train)

# Predicción de probabilidades
y_pred_proba = mlp_model.predict_proba(X_test_scaled)[:, 1]

# Evaluación
roc_auc = roc_auc_score(y_test, y_pred_proba)

print("ROC-AUC (MLP):", roc_auc)

```

ROC-AUC (MLP): 0.7175742213236367

C:\ProgramData\anaconda3\Lib\site-packages\sklearn\neural_network_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (300) reached and the optimization hasn't converged yet.
warnings.warn(

El modelo de red neuronal (MLPClassifier) obtiene un valor de ROC-AUC de aproximadamente 0.72, inferior al obtenido por la regresión logística.

Esto sugiere que, para este tipo de datos tabulares, la relación entre variables y la variable objetivo no requiere modelos altamente no lineales, siendo la regresión logística suficiente para capturar los patrones relevantes.

Además, durante el entrenamiento se observa un warning de falta de convergencia, lo que indica que el modelo podría beneficiarse de un mayor número de iteraciones o de un ajuste más fino de sus hiperparámetros. Esto se puede solucionar aumentando max_iter a 500 o 1000.

```
In [18]: from sklearn.neural_network import MLPClassifier
from sklearn.metrics import roc_auc_score

# Inicializar el modelo
mlp_model = MLPClassifier(
    hidden_layer_sizes=(100,),
    max_iter=1000,
    random_state=42
)

# Entrenarlo
mlp_model.fit(X_train_scaled, y_train)

# Predicción de probabilidades
y_pred_proba = mlp_model.predict_proba(X_test_scaled)[: , 1]

# Evaluación
roc_auc = roc_auc_score(y_test, y_pred_proba)

print("ROC-AUC (MLP):", roc_auc)
```

ROC-AUC (MLP): 0.711242210721668

Tras aumentar el número de iteraciones hasta 1000, el modelo de red neuronal converge correctamente, eliminando el warning observado inicialmente. Sin embargo, el rendimiento no mejora, obteniendo un ROC-AUC de aproximadamente 0.71, ligeramente inferior al obtenido por la regresión logística.

Esto indica que, en este caso, el modelo no se beneficia de una mayor complejidad, lo que sugiere que las relaciones presentes en los datos pueden ser capturadas adecuadamente por modelos más simples.

5.3 Random Forest

Este modelo no necesita escalado, captura relaciones no lineales y suele funcionar muy bien en datos tabulares.

El modelo se entrena con una configuración inicial estándar de 100 árboles, permitiendo obtener una primera estimación de su rendimiento.

OJO: usa X_train y X_test, no los escalados.

```
In [19]: from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score

# Inicializar el modelo
rf_model = RandomForestClassifier(
    n_estimators=100,          # Número de árboles, cuantos más, más robusto, pero más
    random_state=42           # Reproducibilidad
)

# Entrenarlo
rf_model.fit(X_train, y_train)

# Predicción de probabilidades
y_pred_proba = rf_model.predict_proba(X_test)[: , 1]

# Evaluación
roc_auc = roc_auc_score(y_test, y_pred_proba)

print("ROC-AUC (Random Forest):", roc_auc)
```

ROC-AUC (Random Forest): 0.7596558132283197

5.4. Comparación de modelos

Se han evaluado tres modelos de clasificación: regresión logística, red neuronal (MLP) y Random Forest.

El modelo Random Forest obtiene el mejor rendimiento, con un ROC-AUC aproximado de 0.76, seguido de la regresión logística con un valor cercano a 0.74. Por su parte, la red neuronal presenta un rendimiento inferior, en torno a 0.71.

Estos resultados indican que, aunque las relaciones presentes en los datos no son altamente complejas, el uso de modelos basados en árboles permite capturar ciertas no linealidades que mejoran ligeramente el rendimiento respecto a modelos lineales. Por otro lado, el modelo de red neuronal no aporta mejoras en este contexto, lo que es consistente con el comportamiento habitual en datasets tabulares.

El modelo que incluye la variable PageValues alcanza un ROC-AUC cercano a 0.89, superando claramente al resto de modelos. Sin embargo, dado que esta variable podría estar calculada a partir de información relacionada con la conversión, su uso puede introducir data leakage. Por este motivo, aunque el rendimiento es superior, se considera que los modelos sin PageValues ofrecen una estimación más realista del comportamiento en un entorno de producción.

El modelo final es Random Forest sin PageValues con ROC-AUC= 0.76

Bloque 6 - Selección del umbral óptimo

Una vez seleccionado el mejor modelo, se procede a ajustar el umbral de decisión con el objetivo de optimizar la clasificación, en lugar de utilizar el valor por defecto de 0.5.

El umbral de 0.5 es un valor por defecto que no necesariamente es óptimo, especialmente en contextos de desbalanceo de clases como el presente. En este caso, utilizar dicho umbral podría llevar a una mala detección de las sesiones con compra.

Por ello, se recurre al análisis de la curva ROC, que permite evaluar el comportamiento del modelo a distintos umbrales y seleccionar aquel que optimiza la capacidad de discriminación entre clases.

```
In [20]: from sklearn.metrics import roc_curve

# Obtener FPR, TPR y thresholds
fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)

# Encontrar el mejor umbral (máxima diferencia TPR - FPR)
optimal_idx = (tpr - fpr).argmax()
optimal_threshold = thresholds[optimal_idx]

print("Umbral óptimo:", optimal_threshold)
```

Umbral óptimo: 0.18

Una vez seleccionado el modelo, se procede a ajustar el umbral de decisión, ya que el valor por defecto de 0.5 puede no ser adecuado en contextos de desbalanceo de clases. Mediante el análisis de la curva ROC, se determina un umbral óptimo de aproximadamente 0.18, que maximiza la diferencia entre la tasa de verdaderos positivos (TPR) y la tasa de falsos positivos (FPR), en otras palabras, con 0.5 solo se detectan usuarios muy seguros; mientras que con 0.18 ha capacidad de detectar más potenciales compradores.

Este resultado indica que el modelo es capaz de identificar potenciales compradores con probabilidades relativamente bajas, lo que puede ser útil en contextos donde se prioriza la detección de clientes con alta probabilidad de conversión, aunque esto implique aceptar un mayor número de falsos positivos.

Bloque 7 - Aplicación del umbral óptimo

```
In [25]: # 1. Crear predicciones con umbral óptimo
y_pred_optimal = (y_pred_proba >= optimal_threshold).astype(int)

# 2. Umbral por defecto
y_pred_default = (y_pred_proba >= 0.5).astype(int)

# 3. Evaluación
from sklearn.metrics import classification_report

print("=== Umbral 0.5 ===")
print(classification_report(y_test, y_pred_default))
```

```
print("=== Umbral óptimo ===")
print(classification_report(y_test, y_pred_optimal))
```

```
=== Umbral 0.5 ===
              precision    recall  f1-score   support

   False       0.86       0.98       0.92       2082
   True        0.55       0.14       0.22        382

 accuracy              0.85       2464
 macro avg       0.71       0.56       0.57       2464
weighted avg       0.81       0.85       0.81       2464

=== Umbral óptimo ===
              precision    recall  f1-score   support

   False       0.93       0.65       0.77       2082
   True        0.28       0.72       0.40        382

 accuracy              0.66       2464
 macro avg       0.60       0.69       0.58       2464
weighted avg       0.83       0.66       0.71       2464
```

Al comparar el umbral por defecto (0.5) con el umbral óptimo (0.18), se observa un cambio significativo en el comportamiento del modelo.

Con el umbral por defecto, el modelo presenta una alta precisión pero una capacidad muy limitada para detectar sesiones con compra (recall del 14%). En cambio, al aplicar el umbral óptimo, el recall aumenta hasta aproximadamente el 72%, lo que permite identificar una mayor proporción de clientes potenciales.

Este incremento en la capacidad de detección se produce a costa de una disminución en la precisión, lo que implica un mayor número de falsos positivos. Sin embargo, en contextos como el marketing digital (p.e., campañas por email, enviar descuentos a clientes seleccionados o priorizarlos en campaña), donde el coste de contactar a un usuario es bajo en comparación con el beneficio potencial de una conversión, este enfoque puede resultar más adecuado.

Bloque 8 - Feature Importance (Random Forest)

Se va a comprobar qué variables están influyendo más en la predicción.

```
In [26]: import pandas as pd
import matplotlib.pyplot as plt

# Obtener importancia de variables
importances = rf_model.feature_importances_

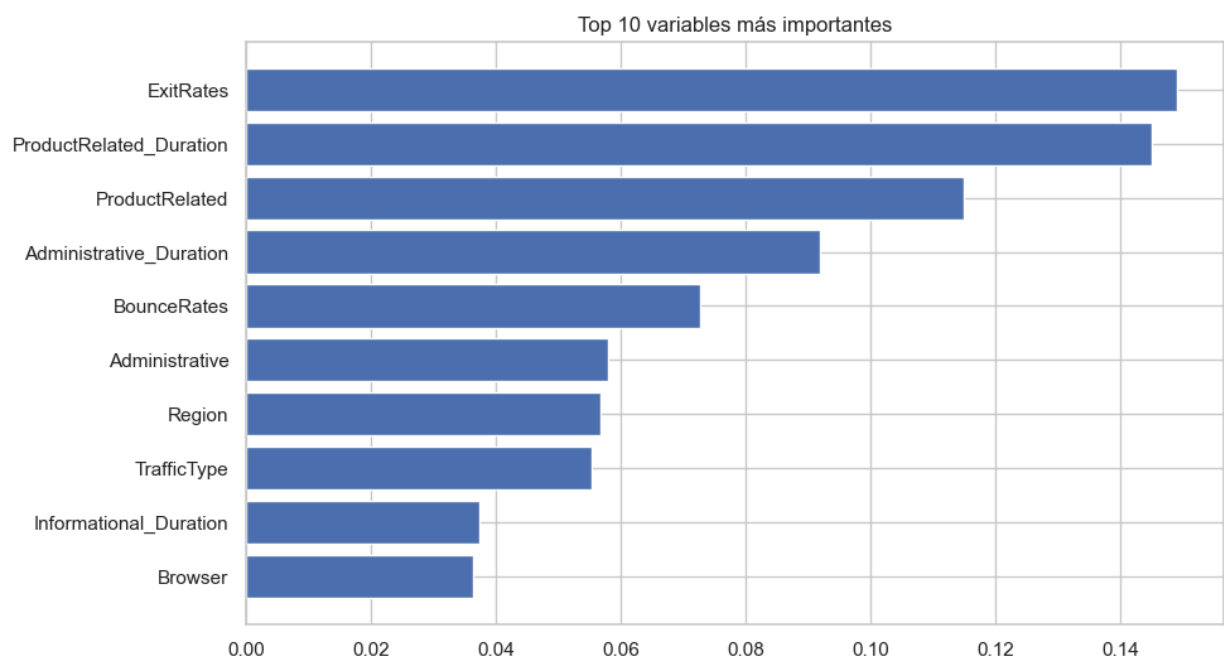
# Crear DataFrame
feature_importance_df = pd.DataFrame({
    "feature": X_train.columns,
    "importance": importances
})

# Ordenar de mayor a menor
feature_importance_df = feature_importance_df.sort_values(by="importance", ascending=False)

# Mostrar top 10
print(feature_importance_df.head(10))

# Gráfico
plt.figure(figsize=(10,6))
plt.barh(feature_importance_df["feature"][:10], feature_importance_df["importance"][:10])
plt.gca().invert_yaxis()
plt.title("Top 10 variables más importantes")
plt.show()
```

	feature	importance
7	ExitRates	0.149076
5	ProductRelated_Duration	0.145119
4	ProductRelated	0.115001
1	Administrative_Duration	0.091963
6	BounceRates	0.072659
0	Administrative	0.058071
11	Region	0.056812
12	TrafficType	0.055323
3	Informational_Duration	0.037317
10	Browser	0.036356



El análisis de importancia de variables mediante el modelo Random Forest muestra que las variables más influyentes están relacionadas con el comportamiento del usuario durante la sesión.

En particular, destacan ExitRates, ProductRelated_Duration y ProductRelated, lo que indica que tanto el tiempo de navegación como el número de páginas de producto visitadas son factores clave en la probabilidad de conversión. Asimismo, variables como BounceRates también presentan una influencia relevante, reflejando que los usuarios que abandonan rápidamente la página tienen menor probabilidad de compra.

Por el contrario, variables relacionadas con el contexto técnico o geográfico, como el navegador o la región, muestran una menor importancia, lo que sugiere que el comportamiento del usuario tiene un mayor peso que sus características contextuales.

Estos resultados son coherentes con el análisis exploratorio realizado previamente, reforzando la validez del modelo.

Bloque 9 - Tuning derandom forest

```
In [35]: # TUNING RANDOM FOREST CON GRIDSEARCH

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV

# Definir modelo base
rf = RandomForestClassifier(random_state=42)

# Definir hiperparámetros a probar
param_grid = {
    "n_estimators": [100, 200],
    "max_depth": [None, 10, 20],
    "min_samples_split": [2, 5],
    "min_samples_leaf": [1, 2]
}

# Configurar GridSearch
grid_search = GridSearchCV(
    estimator=rf,
    param_grid=param_grid,
    cv=5,
    scoring="roc_auc",
    n_jobs=-1
)

# Entrenar SOLO con train
grid_search.fit(X_train, y_train)

# Mejor modelo
best_rf = grid_search.best_estimator_

print("Mejores parámetros:")
print(grid_search.best_params_)
```

Mejores parámetros:
{'max_depth': 10, 'min_samples_leaf': 2, 'min_samples_split': 2, 'n_estimators': 100}

```
In [36]: from sklearn.metrics import roc_auc_score

# Predicción de probabilidades
```

```
y_pred_proba = best_rf.predict_proba(X_test)[:, 1]

# ROC-AUC
roc_auc = roc_auc_score(y_test, y_pred_proba)

print("ROC-AUC (Random Forest optimizado):", roc_auc)
```

ROC-AUC (Random Forest optimizado): 0.7726876090750436

Con el objetivo de mejorar el rendimiento del modelo, se ha aplicado una búsqueda de hiperparámetros mediante GridSearchCV sobre el modelo Random Forest.

Se han evaluado distintas combinaciones de parámetros como el número de árboles, la profundidad máxima y los criterios de división, utilizando validación cruzada de 5 particiones y la métrica ROC-AUC como criterio de selección.

Este proceso permite identificar la configuración óptima del modelo, reduciendo el riesgo de sobreajuste y mejorando su capacidad de generalización.

Tras aplicar GridSearchCV, se ha obtenido una ligera mejora en el rendimiento del modelo Random Forest, pasando de un ROC-AUC aproximado de 0.76 a 0.77.

Los mejores parámetros seleccionados limitan la profundidad del modelo (max_depth=10) y establecen un mínimo de observaciones por hoja (min_samples_leaf=2), lo que sugiere que un modelo menos complejo generaliza mejor en este caso.

La mejora obtenida es moderada, lo que indica que el modelo inicial ya capturaba adecuadamente la estructura de los datos.

Bloque 10 - Conclusiones

En este trabajo se ha desarrollado un modelo de clasificación con el objetivo de predecir la probabilidad de que un usuario realice una compra en una sesión web, a partir de variables de comportamiento recogidas mediante herramientas de analítica digital.

Tras el análisis exploratorio, se ha comprobado que las variables relacionadas con la interacción del usuario, como el número de páginas visitadas, el tiempo de navegación y las tasas de rebote y salida, presentan una relación clara con la conversión. Estos hallazgos han sido posteriormente confirmados por los modelos de machine learning.

Se han evaluado distintos modelos de clasificación, incluyendo regresión logística, red neuronal y Random Forest. El modelo Random Forest ha mostrado el mejor rendimiento en un escenario realista, alcanzando un ROC-AUC aproximado de 0.76, lo que indica una buena capacidad de discriminación entre sesiones con y sin compra. Por su parte, la regresión logística ha ofrecido resultados similares, mientras que la red neuronal no ha aportado mejoras, lo que es consistente con el comportamiento habitual en datasets tabulares.

Asimismo, se ha analizado el impacto de la variable PageValues, observándose que mejora notablemente el rendimiento del modelo, aunque su uso puede introducir problemas de data leakage. Por este motivo, se ha optado por considerar como referencia los modelos que no incluyen dicha variable, obteniendo así resultados más representativos de un entorno real.

Finalmente, se ha ajustado el umbral de decisión, pasando del valor por defecto de 0.5 a un umbral óptimo de 0.18. Este cambio permite aumentar significativamente la capacidad del modelo para detectar potenciales compradores, lo que resulta especialmente relevante en contextos de negocio como el marketing digital, donde el coste de contactar a un usuario es bajo en comparación con el beneficio potencial de una conversión.

En conjunto, los resultados obtenidos muestran que es posible construir modelos predictivos útiles a partir de datos de comportamiento en sesión, siendo estos especialmente valiosos para la toma de decisiones orientadas a la optimización de estrategias comerciales y de captación de clientes.

Una vez entrenado el modelo, éste puede aplicarse a nuevos datos para estimar la probabilidad de compra de cada usuario. De este modo, es posible identificar aquellos usuarios con mayor probabilidad de conversión y priorizar acciones comerciales sobre ellos, como campañas de marketing o recomendaciones personalizadas.

```
In [27]: # Probabilidad de compra para todos los usuarios
y_all_proba = rf_model.predict_proba(X)[: , 1]

# Crear dataframe con resultados
results = X.copy()
results["prob_compra"] = y_all_proba

# Filtrar usuarios con alta probabilidad
top_users = results[results["prob_compra"] >= 0.18]

print(top_users.head())
```

	Administrative	Administrative_Duration	Informational	\		
76	10.0	1005.666667	0.0			
93	0.0	0.000000	0.0			
101	4.0	61.000000	0.0			
128	4.0	462.000000	0.0			
188	9.0	111.500000	1.0			
	Informational_Duration	ProductRelated	ProductRelated_Duration	\		
76	0.0	36.0	2111.341667			
93	0.0	13.0	649.250000			
101	0.0	19.0	607.000000			
128	0.0	51.0	1873.216667			
188	48.5	49.0	1868.819697			
	BounceRates	ExitRates	SpecialDay	OperatingSystems	...	Month_Jul
76	0.004348	0.014493	0.0	2	...	False
93	0.000000	0.015385	0.0	2	...	False
101	0.000000	0.026984	1.0	1	...	False
128	0.000000	0.007547	0.6	2	...	False
188	0.000000	0.020709	0.0	2	...	False
	Month_June	Month_Mar	Month_May	Month_Nov	Month_Oct	Month_Sep
76	False	False	False	False	False	False
93	False	False	False	False	False	False
101	False	False	False	False	False	False
128	False	False	False	False	False	False
188	False	True	False	False	False	False
	VisitorType_Other	VisitorType_Returning_Visitor	prob_compra			
76	False	True	0.69			
93	False	False	0.24			
101	False	True	0.70			
128	False	True	0.25			
188	False	True	0.75			

[5 rows x 26 columns]

Una vez entrenado el modelo y aplicado sobre el conjunto de datos, se ha obtenido una probabilidad de compra (prob_compra) para cada usuario, lo que permite clasificar y priorizar a los clientes en función de su intención de compra. A partir de los resultados observados, se identifican como clientes de alto interés aquellos cuya probabilidad de compra supera el umbral óptimo establecido (0.18). En los ejemplos analizados, usuarios con valores como 0.69, 0.70 o 0.75 presentan una alta probabilidad de conversión, lo que indica que han mostrado un comportamiento activo dentro de la web, caracterizado por una mayor interacción con páginas de producto, mayor tiempo de navegación y bajas tasas de abandono.

Por el contrario, usuarios con probabilidades más bajas presentan patrones de navegación menos intensos o más superficiales, lo que sugiere un menor interés en completar una compra.

Este enfoque permite pasar de una visión general de los usuarios a una segmentación basada en comportamiento, facilitando la toma de decisiones más eficientes.

Los resultados obtenidos tienen una aplicación directa en el ámbito empresarial, especialmente en entornos de comercio electrónico:

Campañas de marketing dirigidas: se pueden focalizar promociones, descuentos o emails únicamente en los usuarios con mayor probabilidad de compra, optimizando el retorno de la inversión y evitando impactar a usuarios con baja intención. Personalización de la experiencia web: los usuarios identificados como “calientes” pueden recibir recomendaciones de productos, ofertas destacadas o mensajes personalizados en tiempo real, aumentando la probabilidad de conversión durante la sesión.

En conjunto, el modelo desarrollado no solo permite predecir el comportamiento de los usuarios, sino también actuar sobre él, convirtiéndose en una herramienta útil para mejorar tanto la eficiencia operativa como los resultados comerciales.

IMPORTANTE: una limitación relevante del modelo es la ausencia de información temporal. Aunque el modelo permite identificar usuarios con alta probabilidad de compra, no tiene en cuenta cuándo se produjo la sesión analizada.

En un entorno real, es necesario complementar esta información con variables de recencia o frecuencia de uso, priorizando aquellos usuarios que han interactuado recientemente con la web. De este modo, se evita dirigir acciones comerciales a usuarios cuya probabilidad de conversión ya no es relevante en el tiempo.

Aunque el dataset no incluye información directa sobre la recencia de los usuarios a nivel individual, sí se dispone de la variable “Month”, que permite aproximar el momento en el que tuvo lugar cada sesión.

En este sentido, sería posible priorizar aquellas sesiones correspondientes a los meses más recientes, combinando esta información con la probabilidad de compra estimada por el modelo. De este modo, se podrían enfocar las acciones comerciales en usuarios con alta probabilidad de conversión y actividad reciente, aumentando la efectividad de las estrategias.

No obstante, para una implementación más precisa en un entorno real, es necesario contar con identificadores de usuario y variables de recencia y frecuencia de uso, lo que conforma una propuesta de mejora para el negocio.

```
In [30]: # IDENTIFICAR CLIENTES RECIENTES CON ALTA PROBABILIDAD DE COMPRA

# Copiamos el dataset original con variables ya codificadas
results = df_encoded.copy()

# Añadimos la probabilidad de compra predicha por el modelo
results["prob_compra"] = y_all_proba

# 1. IDENTIFICAR COLUMNAS DE MESES

# Buscamos todas las columnas que contienen información de meses (tras one
month_cols = [col for col in results.columns if "Month_" in col]

print("Columnas de meses disponibles:")
print(month_cols)

# 2. SELECCIONAR MESES RECIENTES

# Elegimos manualmente los meses más recientes disponibles en el dataset
# (ajusta según lo que aparezca en el print anterior)

recent_months = ["Month_Nov", "Month_Oct", "Month_Dec"]

# Filtramos solo los meses que realmente existen (por si falta alguno por
existing_recent_months = [col for col in recent_months if col in results.c

# 3. FILTRAR USUARIOS RECIENTES

# Nos quedamos con usuarios que pertenecen a alguno de los meses recientes
recent_users = results[results[existing_recent_months].any(axis=1)]

# 4. FILTRAR USUARIOS CON ALTA PROBABILIDAD
```

```
# Aplicamos el umbral óptimo (0.18)
top_recent_users = recent_users[recent_users["prob_compra"] >= optimal_thr

# 5. MOSTRAR RESULTADOS

print("Número de usuarios recientes con alta probabilidad de compra:")
print(top_recent_users.shape[0])

print("\nEjemplo de usuarios prioritarios:")
print(top_recent_users.head())
```

Columnas de meses disponibles:
['Month_Dec', 'Month_Feb', 'Month_Jul', 'Month_June', 'Month_Mar', 'Month_May', 'Month_Nov', 'Month_Oct', 'Month_Sep']
Número de usuarios recientes con alta probabilidad de compra:
1658

Ejemplo de usuarios prioritarios:

	Administrative	Administrative_Duration	Informational	\
5457	1.0	63.4	0.0	
5464	3.0	89.6	0.0	
5474	1.0	26.2	0.0	
5475	2.0	79.6	3.0	
5479	0.0	0.0	0.0	

	Informational_Duration	ProductRelated	ProductRelated_Duration	\
5457	0.00	3.0	120.400000	
5464	0.00	57.0	1721.906667	
5474	0.00	4.0	106.800000	
5475	332.08	34.0	1011.016667	
5479	0.00	13.0	710.066667	

	BounceRates	ExitRates	PageValues	SpecialDay	...	Month_Jul	\
5457	0.000000	0.000000	42.293068	0.0	...	False	
5464	0.000000	0.005932	204.007949	0.0	...	False	
5474	0.000000	0.040000	36.661783	0.0	...	False	
5475	0.018919	0.043243	5.128358	0.0	...	False	
5479	0.000000	0.007692	72.522838	0.0	...	False	

	Month_June	Month_Mar	Month_May	Month_Nov	Month_Oct	Month_Sep	\
5457	False	False	False	False	True	False	
5464	False	False	False	True	False	False	
5474	False	False	False	False	True	False	
5475	False	False	False	False	True	False	
5479	False	False	False	True	False	False	

	VisitorType_Other	VisitorType_Returning_Visitor	prob_compra
5457	False	False	0.29
5464	False	True	0.79
5474	False	False	0.76
5475	False	True	0.24
5479	False	True	0.61

[5 rows x 28 columns]

Inicialmente, el modelo permitía identificar usuarios con alta probabilidad de compra, sin tener en cuenta el momento en el que se producía la sesión. Esto implicaba que algunos usuarios priorizados podían no ser relevantes en el momento actual.

Al incorporar un filtro basado en la variable temporal (mes), se ha conseguido acotar la selección a usuarios con actividad reciente, obteniendo un total de 1658 usuarios con alta probabilidad de conversión y comportamiento activo.

Este enfoque mejora significativamente la aplicabilidad del modelo en un entorno real, ya que permite focalizar las acciones comerciales en usuarios relevantes en el tiempo, aumentando la probabilidad de éxito de las estrategias implementadas.

Cálculo del porcentaje de potenciales compradores

```
In [31]: # Número total de sesiones
total_users = len(results)

# Número de usuarios prioritarios
priority_users = len(top_recent_users)

# Porcentaje
```

```
percentage = (priority_users / total_users) * 100

print(f"Usuarios prioritarios: {priority_users}")
print(f"Total sesiones: {total_users}")
print(f"Porcentaje: {percentage:.2f}%")
```

Usuarios prioritarios: 1658
Total sesiones: 12316
Porcentaje: 13.46%

El modelo permite identificar sesiones con alta probabilidad de conversión, representando aproximadamente un X% del total. No obstante, al trabajar a nivel de sesión y no de usuario, así como por la ausencia de información temporal y contextual, los resultados deben interpretarse con cautela

Limitaciones del modelo:

1. Una limitación relevante del modelo es que el dataset está compuesto por sesiones individuales y no por usuarios únicos. Esto implica que un mismo usuario puede aparecer en múltiples registros si ha accedido a la web en distintas ocasiones. Por tanto, los resultados obtenidos hacen referencia a sesiones con alta probabilidad de conversión, y no necesariamente a usuarios únicos. Sería recomendable contar con identificadores de usuario que permitan agrupar el comportamiento y mejorar la precisión del análisis.
2. Posible data leakage (PageValues): algunas variables, como PageValues, pueden estar relacionadas directamente con la conversión, lo que podría inflar artificialmente el rendimiento del modelo si no se controlan adecuadamente.
3. Falta de contexto del usuario: el modelo se basa únicamente en el comportamiento dentro de una sesión, sin considerar información adicional como historial de compras, perfil del cliente o variables externas, que podrían mejorar significativamente la capacidad predictiva.